

Towards Typed Conversational Interfaces

CHRIS MARTENS, Northeastern University, USA
MANAS AGRAWAL, Northeastern University, USA

Conversational interfaces (CIs) enable human-computer interaction through a dialogue between human and computational agent, typically through natural language. Increasingly, these interfaces are replacing standard front-end user interfaces (e.g. using HTML widgets like drop-down menus and buttons) to offer services such as booking and managing reservations. However, mainstream approaches obscure the relationship between user expressions and their corresponding computational meanings, leading to frustrating interactions and malformed dialogue states.

Our goal is to support CIs with more discoverable affordances, more predictable agent responses, and static guarantees about dialogue states, for which we propose a notion of *type-safe dialogue*. We sketch an approach to authoring service-specific typed interface domains over which a general dialogue language of queries, responses, and requests for information may be given static guarantees through a linear type system.

Additional Key Words and Phrases: conversational interfaces, type systems

ACM Reference Format:

Chris Martens and Manas Aggrawal. 2025. Towards Typed Conversational Interfaces. In *Proceedings of the 15th PLATEAU Workshop on Programming Languages and Human-Computer Interaction (PLATEAU '25)*. 8 pages. <https://doi.org/XX.XXXX/XXXXXXXX.XXXXXXX>

1 Introduction

Conversational interfaces (CIs) enable human-computer interaction through a dialogue between human and computational agent, typically accepting input in the form of typed or spoken natural language. Increasingly, these interfaces are replacing standard front-end user interfaces (e.g. using HTML widgets like drop-down menus and buttons) to offer services such as booking flights, hotels, and appointments; managing purchases or reservations; and mediating customer service requests. Often, usability is cited as motivation: nontechnical users find complex interfaces confusing and would prefer to ask for what they need as they would a human.

In practice, however, the mainstream approach to CIs has flaws. In the example contexts listed previously, the scope of “supported operations” through the interface is closed and well-defined: users have a carefully scaffolded set of interactions with the database that they can perform, such as issuing queries over structured data and associating their name and contact information with a reservation. However, by only showing the user a blank text field prompting a natural language question, they invite arbitrary linguistic expressions that do not map to any supported operation, often resulting in a generic response asking the user to rephrase. Indeed, if a user knows exactly what they want to do, they must *guess* what natural language phrasing will achieve it. In other words, conversational interfaces fail the classical UI principle of **discoverability**. Other challenges arise when users can express their intent in such a way that gets misinterpreted by the system, so it might perform the wrong action or takes them down a dialogue branch they cannot easily reverse: these are examples of what Don Norman might call a *gulf of execution* [7]. Finally, if the

Authors’ Contact Information: Chris Martens, Northeastern University, Boston, MA, USA; Manas Aggrawal, Northeastern University, Boston, MA, USA.



This work is licensed under a Creative Commons Attribution 4.0 International License.
PLATEAU '25, Boston, MA

© 2025 Copyright held by the owner/author(s).
<https://doi.org/XX.XXXX/XXXXXXXX.XXXXXXX>

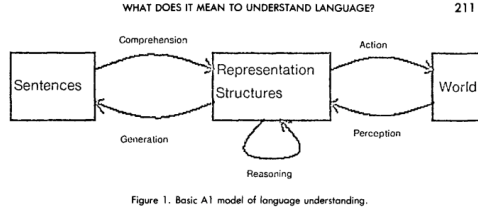


Fig. 1. Terry Winograd’s architecture diagram for computer understanding of natural language.

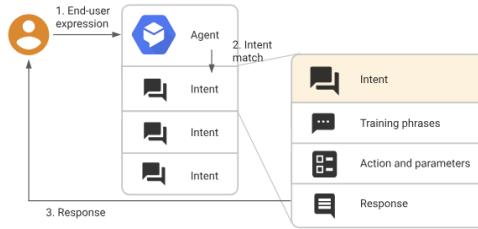


Fig. 2. Google’s architecture diagram for their conversational interface authoring tool, Dialogflow.

system does not perform the expected operation but instead requests clarification or says it cannot understand what they said, they have little to work with for understanding what went wrong, a corresponding *gulf of evaluation* [7].

We contend that these issues can be rectified if we see CIs less as an “AI problem” and more of a “PL+HCI problem”: what’s missing is a precise interface language between human and machine. Some users might still prefer the surface syntax of natural language, but there are sufficiently advanced tools to relegate all of the “fuzzy matching” to the mapping from natural language to structured, unambiguous expressions, at which point computational ideas of syntax, type, and semantics can apply.

On this premise, we propose an approach to authoring conversational interfaces through formal syntax and type systems. We sketch an approach to authoring a service-specific type system through which developers may articulate the space of available user queries. Atop this mechanism we develop a notion of *type-safe dialogue* which uses linearity, in the linear logic sense, to characterize dialogues that may accept incomplete information and resolve that information interactively. We further speculate as to how type systems may support other aspects of CI development, ultimately motivated by achieving a higher standard of mutual intelligibility between humans and machines.

1.1 Precedent and Potential

Humans have seemingly wanted to have natural language conversations with computers since they invented them. To computing researchers as early as Terry Winograd, author of the SHRDLU dialogue system [13], it was apparent that one should not try to operate directly on natural language text, but rather mediate between natural language and symbolic representations of *world models* and *dialogue states* (see figure 1). The idea lives on today in CI authoring systems like Google’s Dialogflow, which gives a visual description of its notion of *intent* using a similar picture to Winograd’s architecture (figure 2).

One area where there has been a striking amount of quiet innovation in dialogue interfaces is digital games, especially genres that grow out of the interactive fiction community. The Inform 7 language for authoring text adventure games, based on Infocom originals beginning in the 1970s, in which natural language input is parsed into a restricted set of commands mapping to game actions like “go north”, “take lantern”, or “give letter to troll”. Non-player characters in these games are often scripted to converse with players, and there is a great deal of theory, writing, and systems both proprietary and open for creating these dialogues (e.g. Inform 7 libraries [12]; Linden Labs’ Versu [4]).

Meanwhile, some subcommunities within programming languages have also reached towards an overlapping set of ideas in concrete syntax such as *structure editors* and *typed holes* to represent partial expressions and well-typed editor states. The modern epitome of these ideas (in the first author’s opinion) is the Hazel programming language implementation [8–10]. Dependently typed languages such as Agda, while not based on structure editing, support interactive development within typed program holes [5, 6], a process that feels not unlike a kind of conversational interface in which human and machine take turns asking for and providing information. These systems collectively inspire the synthesis of mathematical and phenomenological criteria we believe possible and useful to support.

2 Work in Progress: Typed Dialogues

The role we envision a type system playing is twofold: first, we wish to give developers a toolkit via custom domain types to communicate to users the possibility space of valid queries. Second, we wish to provide static guarantees about maintaining well-formed dialogue states during human-agent interaction. We have made a little progress on a formal specification for this idea, which we present here.

2.1 Example Dialogue

Figure 3 depicts a sample dialogue between agent and user with natural language utterances alongside a possible representation for them in abstract syntax. (This image represents an earlier iteration of the system and may not exactly match the syntax described here.) This example is representative of the kind of interaction we aim to support.

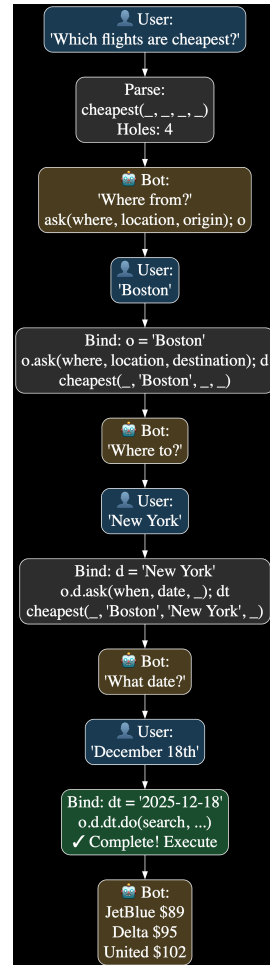


Fig. 3. Internal representation of a dialogue with a chatbot for booking flights.

2.2 Dialogue States

The main concept we aim to formalize is the idea of *partial* queries and actions given by the human interlocutor, which must be resolved by the agent in order to be well-formed and acted upon. We also want to support well-scoped interactions, i.e., the ability to reference entities that arise dynamically as responses to queries.

To support this idea operationally, a dialogue state needs to track entities scoped to the dialogue (e.g., a list of flights from Denver to Boston generated in response to a query), the set of outstanding parameters needed to answer queries or execute actions, and a stack maintaining the work to be done once those parameters are resolved. A dialogue state *computes* by interacting with a *dialogue stream*. We are working towards an operational semantics described by state transitions over tuples $(\mathcal{E}, k, \delta)$ where $\mathcal{E}$ is an environment, k is a stack, and δ is the stream of user and agent dialogue utterances.

In the meantime, we have developed a (work in progress) static semantics for typing dialogue streams. A single well-formedness type *Dia* is meant to characterize dialogues that are well-formed in all dialogue states. Dialogues are sequences of abstract utterances expressed by both the user and the agent, which include queries, requests for missing information, and responses to those requests from the user. We present this system formally next.

2.3 Domain sorts

Our type system is parameterized over a set of domain-specific *entity sorts* s , such as

$$s ::= \text{flight} \mid \text{hotel} \mid \text{restaurant} \mid \text{city} \mid \text{date} \mid \text{person} \mid \dots$$

These sorts constrain the types of questions that can be asked and the domain of permissible values in responses.

2.4 Interrogatives and Queries

User queries look like natural-language questions, beginning with interrogatives like “which”, “when”, “where”, and “how many”. Each interrogative suggests an associated entity sort for the expected response. Semantically, questions beginning with different interrogatives can be elaborated to “which” questions along with their associated type. For example, the question “Where does flight JetBlue 117 fly from?” can be rephrased as “Which *city* is JetBlue 117’s origin?” With this in mind, we can formulate a general typing rule for queries as follows.

$$\frac{s : \text{EntitySort} \quad \Gamma, x : \text{Entity } s \vdash c : \text{Constraint}}{\Gamma \vdash \text{which}(s, x.c) : \text{Query } s} \quad (\text{T-Query})$$

Constraints could draw from any of a number of designs for predicates over domain entities, but a representative choice assuming a database-like backend would be a kind of record pattern. For example, the query above might be encoded as:

$$\text{which}(\text{city}, x. \text{flight}\{\text{id} = \text{jb117}, \text{origin} = x\}) : \text{Query city}$$

Implicitly, the set of parameters given in the constraint may be a subset of the actual fields associated with the entity sort, although some may be required as part of the constraint in order to execute the query. In this case, domain authors would need to provide a mapping between each entity sort and its schema for pattern matching (say, nested labeled sums and records of entities). We elide this particular detail from our example.

2.5 Partial queries

Queries can be issued that are missing information. We capture this idea with function types.

$$\frac{\Gamma, x : \text{Entity } s \vdash q : Q}{\Gamma \vdash \text{partial}(x.q) : s \rightarrow Q} \quad (\text{T-Query-Partial})$$

2.6 Dialogues

Dialogues d have the following syntax:

$\text{ask}(q); u.d$	Ask a query q ; bind it to u and continue as dialogue d
$\mid \text{request}(s, u); r.u'.d$	Request data of sort s ; continue as d
$\mid \text{provide}(v, r); d$	Provide data v for request r ; continue as d
$\mid \text{answer}(u); a.d$	Provide an answer to query u
$\mid \text{done}$	End the dialogue

Dialogues are *linearly typed* over contexts that track information that must be used or resolved, called query contexts. Query contexts Γ may contain:

$x : \tau$	Variables standing for values of domain type τ
$u : Q$	Query variables of type Q
$r : \text{Req } \tau$	Requests for information of type τ

We also may have *answer contexts* Δ that represent answers to queries, consisting of variables $a : \text{Ans } s$. Answer contexts are not treated linearly (a dialogue may end successfully without emptying them). However, given that answer contexts only come up in limited places, we will omit them in rules where they are “threaded through” the rule unchanged.

The typing judgment $\boxed{\Delta; \Gamma \vdash d : \text{Dia}}$ is defined by the following rules.

Asking a query.

$$\frac{\Gamma \vdash q : Q \quad u : Q \vdash d : \text{Dia}}{\Gamma \vdash \text{ask}(q); u.d : \text{Dia}} \quad (\text{T-Dia-Ask})$$

Resolving partial queries.

$$\frac{\Gamma, r : \text{Req } \tau, u' : Q \vdash d : \text{Dia}}{\Gamma, u : \tau \rightarrow Q \vdash \text{request}(\tau, u); r.u'.d : \text{Dia}} \quad (\text{T-Dia-Request})$$

$$\frac{\Delta \vdash v : \tau \quad \Delta; \Gamma \vdash d : \text{Dia}}{\Delta; \Gamma, r : \text{Req } \tau \vdash \text{provide}(v, r); d : \text{Dia}} \quad (\text{T-Dia-Provide})$$

Answering complete queries.

$$\frac{\Delta, a : \text{Ans } s; \cdot \vdash d : \text{Dia}}{\Delta; u : \text{Query } s \vdash \text{answer}(u); a.d : \text{Dia}} \quad (\text{T-Dia-Answer})$$

Note that the query variable must be the *only* member of the context.

Hypotheticals.

$$\frac{}{\Delta, x : \tau \vdash x : \tau} \quad (\text{T-Hyp})$$

Variables of domain type representing answers returned from queries may be referenced to the right of the turnstile. There is no hypothetical or initial rule in our system for the linear context; instead (and in place of standard elimination forms), the typing rules operate directly on these variables.

Concluding the dialogue. To end the dialogue, the query context must be empty.

$$\frac{}{\Delta; \cdot \vdash \text{done} : \text{Dia}} \quad (\text{T-Dia-Done})$$

2.7 Safety Properties

We turn to the question of what static guarantees we can provide using our type system. To specify this, we would need an operational semantics and likely a notion of mapping runtime environments $\mathcal{E}$ to static environments $\Delta; \Gamma$. We also make the simplifying assumption that the whole dialogue is known statically. Then we claim that if the dialogue typechecks according to our rules, it should map to a sequence of program states (including a transcript of the dialogue itself) in which no malformed queries or actions are executed.

These properties may look something like standard progress and preservation:

PROPOSITION 2.1 (PROGRESS). *If $\Delta; \Gamma \vdash d : \text{Dia}$ and environment $\mathcal{E}$ matches $\Delta; \Gamma$, then either:*

- (1) $d = \text{done}$ and $\Gamma = \cdot$ (conversation complete), or
- (2) There exists $\mathcal{E}', e'$ such that $(\mathcal{E}, d) \longrightarrow (\mathcal{E}', d')$ (can take a step)

THEOREM 2.2 (PRESERVATION). *If $\Delta; \Gamma \vdash d : \text{Dia}$ and $\mathcal{E}$ matches $\Delta; \Gamma$ and $(\mathcal{E}, d) \longrightarrow (\mathcal{E}', d')$, then $\Delta; \Gamma \vdash d' : \text{Dia}$.*

We also might want something like termination, indicating that queries will always eventually be resolved:

PROPOSITION 2.3 (RESOLUTION). *If $\Delta; \Gamma \vdash d : \text{Dia}$ and $\mathcal{E}$ matches $\Delta; \Gamma$, then there exists $\mathcal{E}'$ such that $(\mathcal{E}, d) \longrightarrow^* (\mathcal{E}', \text{done})$*

3 Future work

There are a number of avenues of future work that seem partial to explore. Here are a few examples of features that most chatbots support that we believe could also be captured through types:

Canceling or editing partial queries. Something like exceptions and/or a more complex notion of dialogue state could support a user “backing out” of previous decisions in the dialogue. We could alternatively default to a mode in which the query is actively being “edited” (including replaced or incrementally modified) and require explicit user confirmation to execute the query. A simple exception mechanism could suffice, but a richer notion like algebraic effect handlers [11] might allow us to support surfacing to the user a more complex language of edits on the conversation state.

Type safety for streaming dialogues. Given that in practice the full dialogue is, in practice, not known statically, we might instead wish to state a finer-grained type safety theorem that guarantees for any *intermediate* dialogue state awaiting input, and any well-typed user input compatible with that state, their interaction can take a step. Such a type system might borrow ideas from linear session types [1] or stream types [2].

Custom agent scripting. Currently the presumed operational semantics of our language *deterministically* generate agent responses, such as requests for clarification, based on resolving ambiguity as defined by our type system. In some cases, of course, developers might want more control, in which case they would author a *scripted agent* that branches on user input and replies differently in different situations. Designing a type system for scripted agents is a natural step to consider, but would likely require an altogether new set of type system design principles. Again, session types may offer some promise. We also consider Lucas Dixon’s work on linear logic for dialogue

planning [3] to be an intriguing possibility for some kind of automated generation of branching dialogues that achieve user goals.

Implementation. The type system currently exists as an on-paper formulation. We expect it would translate very nicely into an encoding via dependent types, in which example domains could also readily be authored, and the stated properties could be mechanized.

Proving out this idea in a way that directly compares to mainstream chatbots would require interfacing with nice surface syntax; if not directly natural language text, at least something that “looks less like code,” i.e. fewer bindings and type annotations, support for general interrogatives, etc. This likely involves a couple of steps:

- (1) An intermediate “concrete syntax” and elaboration/type reconstruction algorithm that turns underspecified expressions in that syntax into explicit bindings, annotations, and complete terms.
- (2) Integration with something like Google Dialogflow to train on natural language input, integrate with a knowledge base, and deploy.

4 Conclusion

We have presented work in progress on a type system to support well-formed dialogues between humans and machines, alongside a process for domain authoring through which developers may surface permissible queries and possible answers through a language that is both precise and, through elaboration of concrete syntax, possible to make layperson-readable. Our long-term goal is to support CIs with more discoverable affordances, more predictable agent responses, and static guarantees about dialogue states.

Potential domains of application. Beyond the corporate-friendly scope of supporting customer interactions with businesses, we believe a more humane approach to CIs really does have potential for usability. Where we think these tools shine is for *interacting with knowledge bases*, which encompasses things like (traditional) libraries, archives, wikis, or other metadata-rich digital stores.

While “everyday user” is a common target audience for conversational audiences, we also speculate that even more technical users, such as programmers, might benefit from them “done right” (and indeed, there is precedent for this in the form of structure editing and interactive dependently-typed programming). In particular, we would like to consider the use case of programmers looking for information within code editors, where the knowledge base in question might be the set of libraries available for the language, documentation for that language or its libraries, or the codebase in the developer’s own project. (It is not by coincidence that these kinds of queries are also common use cases of editor-integrated large language models.)

References

- [1] Luís Caires and Frank Pfenning. 2010. Session types as intuitionistic linear propositions. In *International Conference on Concurrency Theory*. Springer, 222–236.
- [2] Joseph W Cutler, Christopher Watson, Emeka Nkurumeh, Phillip Hilliard, Harrison Goldstein, Caleb Stanford, and Benjamin C Pierce. 2024. Stream types. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 1412–1436.
- [3] Lucas Dixon, Alan Smaill, and Tracy Tsang. 2009. Plans, actions and dialogues using linear logic. *Journal of Logic, Language and Information* 18, 2 (2009), 251–289.
- [4] Richard Evans and Emily Short. 2013. Versu—a simulationist storytelling system. *IEEE Transactions on Computational Intelligence and AI in Games* 6, 2 (2013), 113–130.
- [5] Conor McBride. 2013. Dependently typed metaprogramming (in Agda). *Lecture Notes* (08 2013). <https://raw.githubusercontent.com/pigworker/MetaprogrammingAgda/master/notes.pdf>
- [6] Ulf Norell. 2013. Interactive programming with dependent types. In *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming*. 1–2.
- [7] Don Norman. 2013. *The design of everyday things: Revised and expanded edition*. Basic books.

- [8] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live Functional Programming with Typed Holes. *PACMPL* 3, POPL (2019). doi:10.1145/3290327
- [9] Cyrus Omar, Ian Voysey, Michael Hilton, Jonathan Aldrich, and Matthew A. Hammer. 2017. Hazelnut: A Bidirectionally Typed Structure Editor Calculus. In *44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*.
- [10] Cyrus Omar, Ian Voysey, Michael Hilton, Joshua Sunshine, Claire Le Goues, Jonathan Aldrich, and Matthew A. Hammer. 2017. Toward Semantic Foundations for Program Editors. In *Summit on Advances in Programming Languages (SNAPL) (LIPICs, Vol. 71)*. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 11:1–11:12.
- [11] Gordon Plotkin and Matija Pretnar. 2009. Handlers of algebraic effects. In *European Symposium on Programming*. Springer, 80–94.
- [12] Emily Short. 2011. NPC conversation systems. *IF Theory Reader* (2011), 331.
- [13] Terry Winograd. 1971. *Procedures as a representation for data in a computer program for understanding natural language*. Technical Report.